scientific reports

Check for updates

OPEN

Securing the CAN bus using deep learning for intrusion detection in vehicles

Ritu Rai[1], Jyoti Grover[1,4], Prinkle Sharma[2,4]✉ & Ayush Pareek[3,4]

The Controller Area Network (CAN) bus protocol is the essential communication backbone in vehicles within the Intelligent Transportation System (ITS), enabling interaction between electronic control units (ECUs). However, CAN messages lack authentication and security, making the system vulnerable to attacks such as DoS, fuzzing, impersonation, and spoofing. This paper evaluates deep learning methods to detect intrusions in the CAN bus network. Using the Car Hacking, Survival Analysis, and OTIDS datasets, we train and test models to identify automotive cyber threats. We explore recurrent neural network (RNN) variants, including LSTM, GRU, and VGG-16, to analyze temporal and spatial features in the data. LSTMs and GRUs handle long-term dependencies in sequential data, making them suitable for analyzing CAN messages. Bi-LSTMs enhance this by processing sequences in both directions, learning from past and future contexts to improve anomaly detection. Our results show that LSTM achieves 99.89% accuracy in binary classification, while VGG-16 reaches 100% accuracy in multiclass classification. These findings demonstrate the potential of deep learning techniques in improving the security and resilience of ITS by effectively detecting and mitigating CAN bus network attacks.

Abbreviations

Bi-LSTM	Bi-Directional LSTM
CAN	Controller Area Network
CNN	Convolutional Neural Network
DL	Deep Learning
ECU	Electronic Control Unit
GRU	Gated Recurrent Unit
IDS	Intrusion Detection System
ITS	Intelligent Transportation System
LSTM	Long-Short Term Memory
ML	Machine Learning
OBD-II	On-Board Diagnostic-II
RNN	Recurrent Neural Network
VGG	16 Visual Geometry Group 16
UTC	Universal Time Coordinated

An intra-vehicle network refers to the interconnected system of electronic components and communication protocols within a vehicle[1]. It allows different components and systems within the vehicle to communicate and share information. In modern vehicles, various electronic systems are responsible for functions such as engine control, transmission control, braking, steering, infotainment, climate control, and more. These systems comprise numerous electronic control units or modules that perform specific tasks. The in-vehicle network facilitates communication between these ECUs, enabling them to exchange data and coordinate their actions. Serving as the communication backbone of the vehicle, the network ensures that different components can interact and work together effectively. This coordination is vital for efficient control, monitoring, and integration of various vehicle functions, contributing to the overall performance, safety, and intelligence of modern transportation systems. In the context of ITS, ensuring the security and integrity of this intra-vehicle network is crucial, as

[1]Department of Computer Science and Engineering, Malaviya National Institute of Technology, Jaipur 302017, India. [2]Information Security and Digital Forensics Department, University at Albany - State University of New York, Albany, New York 12222, USA. [3]Goldman Sachs, Helios Business Park, Service Rd, Kadabeesanahalli, Bengaluru, Karnataka 560103, India. [4]Jyoti Grover, Prinkle Sharma, and Ayush Pareek: These authors contributed equally to this work. ✉email: psharma2@albany.edu

vulnerabilities in the system can lead to serious security risks and operational failures[2]. There are various intra-vehicle network protocols, and the selection of them relies on attributes such as the complexity of the vehicle's systems, data transfer requirements, and cost considerations.

Among the several network protocols, CAN is mainly used for intra-vehicle communication because of its many benefits, including affordability, speed, lightweight, resilience, and ease of installation[2]. CAN enables the exchange of crucial control data between numerous ECUs on a vehicle[3]. The implementation of a particular control system function, such as the throttle, brake, or steering, is carried out by each ECU. The subdivision of ECUs is based on their intended function or communication speed, with multiple gateways connecting the subnets through the in-vehicle CAN, providing reliable, efficient, and cost-effective communication[4–6]. Initially, no security aspect was considered when developing the CAN protocol. Malicious hackers can use this CAN protocol vulnerability to launch assaults inside a vehicle's network. As a result of the CAN protocol, the in-vehicle network has become susceptible to external and internal cyber threats[7]. Modern vehicles have more entry points than older ones, which increases the potential for attacks on intra-vehicle networks[8]. The expansion of intrusions proves that the protocol is susceptible to automotive cyberattacks.

CAN is susceptible to attacks due to the absence of encryption and verification. Even if cryptographic approaches are the straightforward approach, it is not possible to implement such algorithms for CAN in automobile systems due to restricted assets (bandwidth and computational power), and the real-time and deterministic behavior of the CAN bus communication. Due to high complexity or backward incompatibility, researchers have demonstrated that current cryptography algorithms are unsuitable for commercial automobiles[9].

Extensive research has explored security vulnerabilities in automotive safety systems. However, conventional cybersecurity mechanisms, such as public key infrastructure (PKI) and message authentication codes (MACs)[10], signature-based[11] are difficult to implement directly in in-vehicle CAN networks due to the limited computational capacity of ECUs and strict real-time constraints. As a result, intrusion detection systems (IDS) have emerged as a viable solution for enhancing CAN bus security[12]. IDSs detect various cyber threats by analyzing both physical attributes (e.g., frequency and voltage)[13,14] and digital attributes (e.g., CAN identifiers)[15].

Deep learning (DL) has transformed artificial intelligence by enabling models to identify complex patterns from large datasets[12,16,17]. In the realm of CAN bus IDS, deep learning plays a crucial role in detecting cyber threats by analyzing CAN traffic patterns[18,19]. Given the high-dimensional nature of CAN data, models like Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) can effectively identify anomalies. By leveraging deep learning, CAN bus IDS can achieve high detection accuracy, adapt to evolving attack patterns, and provide real-time threat mitigation, thereby strengthening the cybersecurity of modern vehicles[20,21]. In this paper, we explore the benefits of DL models in IDS to identify intrusions in the CAN bus communication. We employed different forms of RNNs, including Long-Short Term Memory (LSTM), Gated Recurrent Unit (GRU), and Bi-directional LSTM, to capture the temporal characteristics within the dataset. Additionally, we employed VGG-16, a variant of CNN, to analyze the spatial attributes.

Motivation

- CANs are vulnerable to cyber attacks due to insufficient security procedures. These attacks' sophistication is always evolving, and new attacks emerge over time[22,23].
- Weak security mechanisms and attacks targeting the CAN bus can result in system malfunctions, posing a severe risk to human life[24].
- While IDS technologies have advanced significantly in recent years, they still struggle to detect all probable attacks in real-world circumstances[25].

Contribution

The following are the contributions made by this paper:

1. Our methodology used the car hacking dataset, survival analysis dataset, and the OTIDS dataset. We combined all three datasets into one single dataset to address all the possible intrusions affecting the CAN bus network.
2. We employed RNN variants: LSTM, GRU, Bi-directional LSTM, and VGG-16 model to learn the temporal and spatial relationship of the dataset and to make the models robust in identifying intrusions.
3. We assessed the performance of deep learning architectures for binary and multiclass labels by applying evaluation metrics to both individual datasets and the combined dataset.

Organization

The organization of the subsequent parts of the article is given as follows: Section Overview and security threats to CAN bus details the outline of CAN and the security threats related to the CAN bus. Section Related works gives a brief description of the related works on CAN bus IDS. Section Dataset description describes the details of the publicly available CAN bus dataset utilized in this work. Section Description of deep learning models presents the description of the structure of the DL models. Section Methodology describes the working of the proposed methodology. Section Experimental setup and results details the hardware and software specifications and compares the proposed technique's performance for individual and combined datasets. Finally, Section Conclusion and future work outlines the paper's findings and suggests further potential research areas.

Overview and security threats to CAN bus

The continuous demand for precise actuators, advanced sensors, and high-quality ECUs in the automotive industry stems from the swift advancements in mechanical and communication technologies. As additional

hardware components are integrated, the complexity of these subsystems in vehicles also grows. To meet the requirements set by the industry, the CAN bus serves as the traditional intra-vehicle automobile network, enabling vehicles to minimize wiring complications and benefit from simplified design[26].

Controller area network (CAN)

The CAN protocol is a popular means of interaction for distributed control systems, especially in automobile and industrial settings. It is developed to allow multiple ECUs to converse with each other efficiently and reliably.

The CAN network is segregated into high-speed and low-speed buses, which are determined by the data rates they support. The bit rate of the high-speed CAN bus spans from 125 Kbps to 1 Mbps, and the low-speed CAN bus runs between 5 and 125 Kbps. The CAN bus is capable of transmitting payloads of up to 8 bytes. The high-speed CAN bus connects time-crucial ECUs such as engine and gearbox control units, whereas the low-speed CAN bus connects ECUs such as door and light control units[2].

Within a CAN bus, a multitude of messages encompassing crucial details such as steering angle, speed, and RPM are transmitted across the whole network to uphold coherence. Each communication on the CAN bus possesses a distinctive identification known as the CAN ID, characterized by either an 11-bit or 29-bit identifier. The base ID segment encompasses the 11-bit identifier, while the extended ID incorporates the rest of the 18 bits. Devices conforming to CAN 2.0A exclusively utilize the base ID, and CAN 2.0B devices employ both ID domains. By leveraging the CAN ID, ECUs can effectively ascertain the relevance of received messages and filter out extraneous signals[25].

A CAN bus data frame is the fundamental unit of information exchange within a CAN network. It is a structured message format used for communication between ECUs. The CAN data frame comprises several fields that carry the necessary information for transmitting and receiving data[25]. The explanation of each field of the CAN message frame is given below and also shown in Fig. 1.

1. **Start of Frame (SOF):** This field serves as an indicator for the initiation of a CAN message and it is of 1 bit.
2. **Arbitration ID:** The arbitration ID selects the preference of the message to be sent and it is 11 bits. Generally, the lower CAN ID means the message is of high priority.
3. **Data Length Code (DLC):** The size of a payload is defined by DLC, which spans from 0 to 4 bits.
4. **Data Field:** The Data field carries the details of the CAN message that is being sent and it is 64 bits.
5. **Cyclic Redundancy Check (CRC):** The purpose of the 16-bit CRC field is to identify transmission faults within messages. It encompasses the CRC series, spanning from SOF to the Data Field.
6. **Acknowledge (ACK):** This field is utilized to determine whether the CAN message was properly acquired by the receiver node. In the event of detecting a transmission error, the sender has the option to make another attempt at transmitting the CAN message. The ACK field, consisting of 2 bits, facilitates this process.
7. **End of Frame (EOF):** The termination of the active CAN is determined by EOF, which spans 7 bits.

Attacks on CAN bus

The main issue with the CAN protocol from a security standpoint is the absence of security measures like encryption[8] and authentication[3]. Due to its broadcasting nature and lack of authentication, the CAN bus network enables any node to connect and intercept all messages. This vulnerability allows adversaries to easily capture CAN bus data[27]. The access of in-vehicle data provides an opportunity for attackers to analyze the specific vehicle's CAN data and execute advanced message infusion attacks, ultimately gaining authority over the targeted vehicle[28]. The attacker can establish a physical link to the CAN bus via an OBD-II port or through wireless technologies like Bluetooth, Wi-Fi, and telecom facilities such as 4G/5G and long-term evolution (LTE). These intrusion vectors serve as vulnerable points to access the CAN bus of the targeted vehicle[2]. Various types of attacks have been discussed below[27,29–32]:

1. **Denial of Service (DoS) Attack:** The intruder injects high precedence CAN bus payloads at regular intervals during this attack. The main objective of a DoS attack is to disrupt network availability. By utilizing the arbitration phase, the ECU that sends a message with the topmost priority CAN ID gains control of the bus, causing delay for other ECUs to transmit their messages. This prevents them from being able to share data effectively. Fig. 2(a) represents the DoS intrusion on the CAN bus. Malicious ECU transmits the CAN ID 0x000, and since the priority of CAN ID 0x000 is high, the transmission of CAN ID 0x123 sent by ECU B is delayed.
2. **Fuzzy Attack:** In the fuzzy attack, Randomly generated IDs and arbitrary payloads are transmitted at an increased frequency, dominating the communication channel. These injected messages overshadow legitimate ones, disrupting the vehicle's normal operations. Fig. 2(b) shows the fuzzy attack. CAN IDs 0x212 and 0x228 are randomly generated by malicious ECU to delay the transmission of CAN ID 0x567, which is meant to be sent by ECU B.

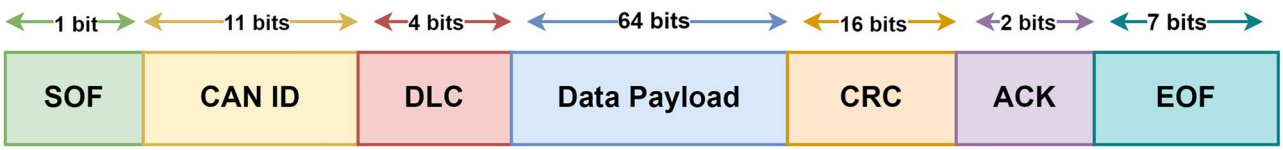

Fig. 1. Structure of CAN message frame.

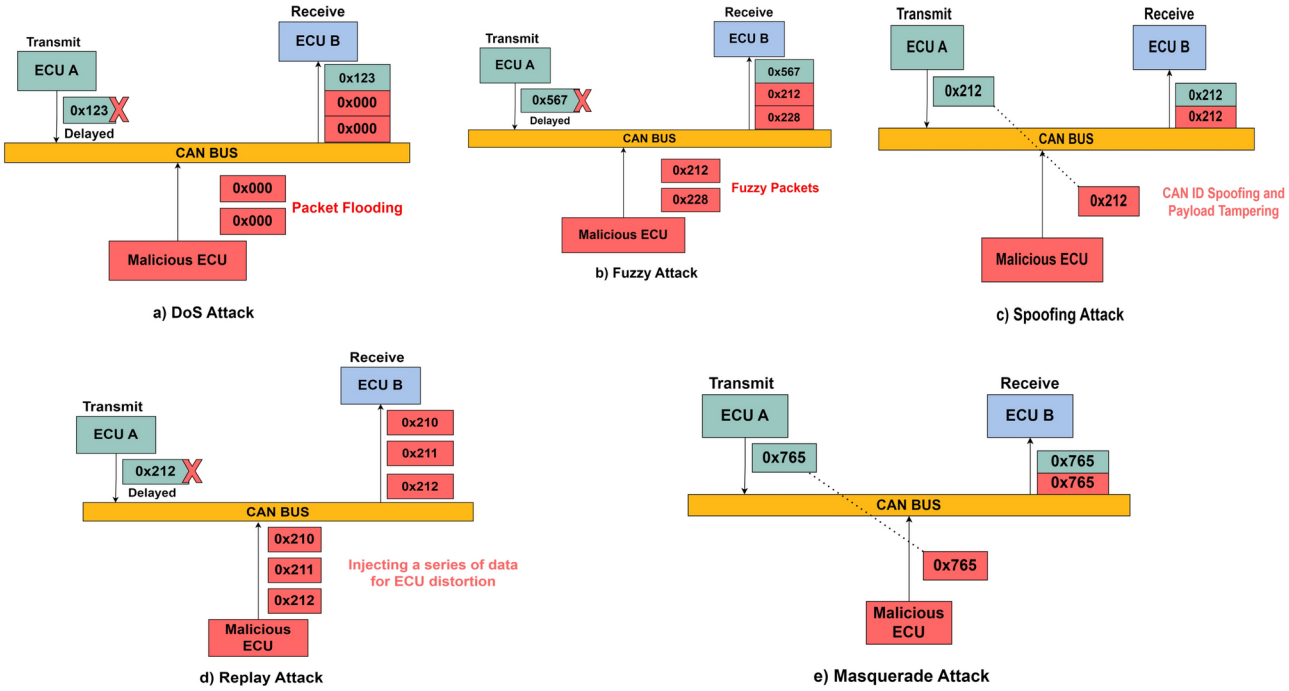

Fig. 2. Attacks on CAN bus.

3. **Spoofing Attack:** In the spoofing attack, the intruder focuses on particular CAN IDs to infuse harmful messages. Fig. 2(c) demonstrates the spoofing attack. Malicious ECU launches a spoofing attack on ECU A's CAN ID 0x212. This attack exploits the lack of authentication and integrity mechanisms in the CAN protocol, allowing malicious actors to deceive ID or payload and manipulate system behavior.
4. **Replay Attack:** Replay attacks involve the attacker recording valid messages and then re-sending them later. While this attack may be relatively simple to execute, it can result in significant security hazards for the vehicle and its passengers. Fig. 2(d) shows the replay attack. The intruding ECU sends the CAN IDs of ECU A. Since the retransmitted messages appear to be legitimate, the targeted ECUs may process them as if they were newly generated, potentially leading to unauthorized actions like unlocking doors, adjusting speed, or interfering with safety-critical systems.
5. **Masquerade Attack:** In this attack, a compromised node pretends to be another node within the network. An instance of this attack involves the attacker observing and gaining knowledge about message IDs and their frequencies from a vulnerable node B as shown in Fig. 2(e). Subsequently, the attacker interrupts the transfer of messages from node A, allowing another ECU to send a falsified message that appears to originate from node A. Despite this substitution, the frequency of messages associated with node B remains unchanged, while malicious ECU assumes the role of the transmitter.

Related works

To enhance vehicle protection and prioritize the safety of drivers and travelers, numerous security techniques and solutions are currently under development and investigation by industry experts and researchers. In current times, various vehicle issues have been uncovered, and substantial research has focused on developing algorithms to identify intrusions on these. Table 1 provides an overview of the existing CAN bus IDS.

Javed et al.[3] developed CANitelliIDS, a hybrid deep learning-based method for intra-vehicle intrusion detection. Using a CAN bus, CANintelliIDS utilizes a hybrid model comprising a CNN and an attention-based GRU to identify various intrusion threats, including both individual and mixed types.

Cheng et al.[4] suggested a temporal convolutional network (TCAN) with global attention to constructing the TCAN-IDS architecture for intra-vehicle intrusion detection. Especially, the TCAN-IDS architecture consistently encrypts 19-bit message attributes, including a priority bit and data field, into a message matrix. The feature extraction model is then employed to extract spatial-temporal detailed characteristics from this matrix. Global attention is utilized to focus on critical regions globally, taking into account channel and spatial feature coefficients, effectively disregarding insignificant byte variations. Lastly, an aberrant traffic monitoring component employs a two-class categorization approach to identify anomalies.

Lo et al.[8] devised a hybrid IDS called HyDL-IDS, that utilizes DL techniques to accurately analyze CAN network traffic through a spatial-temporal representation. To do this, the authors used a sequence of CNN and LSTM fusions to extract both spatial and temporal characteristics from network data.

Hossain et al.[33] proposed an LSTM-based IDS to uncover and counter CAN bus network attacks. To train and test the model, the authors created their dataset by first gathering benign data from a vehicle, then deliberately inserting attacks and gathering additional data.

Year and Author	Dataset Used	Methodology Applied	Limitations
Javed *et al.*[3] 2021	OTIDS	The implementation involves utilizing a blend of CNN and attention-based GRU to detect both individual and mixed instances of intrusions	The dataset used for evaluation is limited to a single vehicle model.
Cheng *et al.*[4] 2022	CAN Signal Extraction and Translation Dataset	An IDS employing an in-vehicle global attentional temporal convolutional network is employed to address the issues associated with real-time monitoring and accurate portrayal of atypical characteristics	During the training phase, the model's filter size and dilation factor are fixed, which limits its adaptability for detecting a broader range of attacks.
Hossain et al.[33] 2020	NAIST CAN Attack and Survival Analysis Dataset	LSTM based IDS is employed to identify and prevent CAN bus network intrusions	The integration of the deep learning-based IDS with the actual CAN bus system is not discussed.
Ma et al.[34] 2022	Car Hacking Dataset	GRU based IDS is developed to deploy inside the vehicle to achieve the real-time intrusion detection	The proposed approach is evaluated on a single dataset, which limits the model's ability to generalize effectively.
Bari et al.[35] 2023	OTIDS and In-vehicle Network Intrusion Detection Dataset	ML-based IDS using SVM, KNN and DT is employed for detection of in-vehicle attacks	Use of single dataset often exhibit a significant imbalance between attack-free data and attack data.
Nguyen et al.[36] 2023	Car Hacking, Survival Analysis, and In-vehicle Network Intrusion Detection Dataset	TAN based IDS for multiclass intrusions. Detection of replay attacks by combining sequential CAN IDs	The experiments were conducted using an extracted dataset. Classification becomes more challenging when the model is deployed in real-time vehicle environments.

Table 1. Overview of CAN bus intrusion detection systems.

Ma et al.[34] focused on a quickly deployable IDS for use within a vehicle. They designed a feature extraction algorithm with less complication to achieve real-time detection and paired it with a lightweight neural network based on GRU to optimize performance. The experimentation involved in-vehicle embedded devices and utilized publicly available datasets. Additionally, the authors explore the potential for integrating this IDS with Over-The-Air (OTA) services to enhance vehicular operating system intelligence and prevent attacks.

Bari et al.[35] introduced an ML-based IDS based on SVM, Decision Tree (DT), and k-NN and evaluated its performance utilizing various real-world datasets. The IDS is trained and tested to detect and categorize intrusion on different automotive datasets (Kia Soul and Chevrolet Spark).

Nguyen et al.[36] devised a multi-class IDS for CAN bus that employs a transformer-based attention network (TAN). The model extends the self-attention process by eliminating RNNs and categorizing attacks into various groups. Additionally, by aggregating sequential IDs, the proposed models may identify replay assaults. The key feature is that, despite the use of consecutive CAN IDs, the architecture can recognize incursion messages without the need for message labeling. Finally, TAN uses transfer learning to enhance the output of architectures learned on tiny data from other automobile architectures by inheriting the benefits of transformers.

Park et al.[37] introduced G-IDCS, a graph-based IDS and categorization method that aims to enhance the safety of the CAN protocol. The method integrates a threshold-based IDS with an ML-based attack classifier. G-IDCS's threshold-based intrusion detection technique minimizes the number of CAN messages needed for identification by greater than 0.0333 and enhances overall attack detection accuracy by more than 9%. The threshold classifier is resilient to alterations attacks and can provide explanations for attack detection characteristics.

Sharmin et al.[38] compared four statistical and two ML-based CAN IDS methods to the Real ORNL Automotive Dynamometer (ROAD) dataset. The ROAD includes the most subtle types of targeted ID forged intrusions, that are harder to investigate including Masquerade attacks. In addition to basic metrics methods, the authors employ balanced accuracy, informedness, markedness, and Matthews correlation coefficient, to give equivalent importance to positive and negative labels and provide more accurate estimates of detection ability, particularly for imbalanced datasets.

Tariq et al.[39] proposed CANTransfer, an intrusion identification approach for the CAN bus that employs transfer learning, in which a Convolutional LSTM architecture is developed using existing intrusions to identify recent intrusions. The architecture can be accommodated to find new invasions with a smaller number of fresh datasets by using one-shot learning.

Al-Jarrah et al.[22] presented the development of an IDS that utilizes ML algorithms to identify new cyber-attacks in CANs. The proposed IDS employs Recurrence Plot (RP) analysis to extract high-level characters from the temporal properties and message dependencies of CAN payloads transmitted on the bus. These features are catered into a custom-designed neural network that is developed to recognize and classify new types of intrusions.

Zhang et al.[40] introduced an IDS model to safeguard the CAN bus and prevent harmful intrusions on vehicles. The IDS integrates conventional rule-based IDS methods with recent ML techniques to stabilize detection precision and efficiency. The effectiveness and efficiency of the suggested approach have been evaluated using datasets gathered from four vehicles with varying models and manufacturers: the Honda Accord, Honda Civic, Ford Fusion, and Chevrolet Volt. Based on the performance of the proposed IDS, it is divided into three categories: strong adversary, medium adversary, and weak adversary.

Wei et al.[41] introduced Domain Adversarial Neural Network (DANN)-based IDS that utilizes the domain adversarial principle to identify variant attacks on in-vehicle networks. By exploiting the domain uniformity between the origin and target domains, the authors were able to extract crucial attack features, which facilitated the successful detection of variant attacks. The authors developed the dataset from real vehicles and performed three types of attacks using the OBD-II port. The attacks employed in the dataset were DoS, Fuzzy, and Tamper attacks. In addition, they have employed a feature fusion technique to extract data features, which significantly enhanced the system's ability to detect variant attacks.

Dataset	Attacks Covered	Labels	Vehicles
Car Hacking[43]	Dos, Fuzzy, RPM Spoofing, Gear Spoofing	Yes	Hyundai Sonata
OTIDS[27]	DoS, Fuzzy, Impersonation	No	KIA Soul
Survival Analysis[29]	Flooding, Fuzzy, Malfunction	Yes	Hyundai Sonata, KIA Soul, Chevrolet Spark

Table 2. Summary of CAN bus datasets.

Attack Name	Total Messages	Normal Messages	Attack Messages
DoS Attack	3,665,771	3,078,250	587,521
Fuzzy Attack	3,838,860	3,347,013	491,847
RPM Spoofing Attack	4,61,702	3,996,805	654,897
Gear Spoofing	4,443,142	3,845,890	597,252
Benign	988,987	988,987	–

Table 3. Overview of car hacking dataset.

Attack Name	Total Messages
DoS Attack	656,579
Fuzzy Attack	591,990
Impersonation Attack	995,472
Benign	2,369,868

Table 4. Overview of OTIDS dataset.

Desta et al.[42] proposed Rec-CNN that employs a CNN that can analyze reiterative images produced from the encrypted classes of CAN frame IDs. The CNN is enabled to occupy the temporal reliance within the series of frame IDs. Utilizing recurrence plots can effectively detect intrusions in the system.

While the aforementioned IDS solutions promise to address CAN's vulnerability by identifying malicious in-vehicle data, current IDSs have a significant problem in their sensitivity to specific kinds of attacks. To handle all possible attacks affecting the CAN bus network, we used three publicly available datasets and aggregated all three datasets into a single file. To understand the temporal and spatial features of the datasets and make the models robust in identifying every kind of intrusion, we used RNN variants: LSTM, GRU, and Bi-directional LSTM and VGG-16 models. The models' efficiency was tested on both binary and multiclass labels.

Dataset description

In this section, we provide a detailed description of the datasets used in the proposed approach. The overall summary of the datasets is provided in Table 2.

1. **Car Hacking Dataset**

In 2018, the Hacking and Countermeasures Research Lab (HCRL) created the car hacking dataset which includes multiple intrusions, such as DoS attacks, fuzzy attacks, and drive gear/RPM gauge spoofing. To capture CAN traffic during message injection attacks, the datasets were generated by accessing the OBD-II port of an actual car. In individual data files, there are 300 occurrences of message injection intrusion, and each intrusion persists for a duration ranging from 3 to 5 seconds. Additionally, the complete duration of CAN traffic in individual datasets ranges from 30 to 40 minutes. The description of each attack is described below:

- **DoS Attack:** CAN ID of 0x000 were injected at every 0.3 milliseconds.
- **Fuzzy Attack:** Random CAN ID and payload were infused at every 0.5 milliseconds.
- **RPM Spoofing:** CAN ID 0x316 was injected at every 1 millisecond.
- **Gear Spoofing:** CAN ID 0x43F was injected at every 1 millisecond.

Table 3 summarizes the normal and attack data instances in each file of the dataset.

2. **OTIDS Dataset**

The dataset comprises different attack types, including DoS attacks, fuzzy attacks, impersonation attacks, and attack-free states, as described in Table 4. Specifically, the researchers used the KIA SOUL to obtain the in-vehicle data.

- **DoS Attack:** CAN ID of 0x000 was injected.

- **Fuzzy Attack:** Act of injecting messages involves sending data with random spoofed CAN ID and payload values.
- **Impersonation Attack:** This attack involves infusing messages that impersonate a node, with an ID of '0x164'.

3. **Survival analysis dataset**

This dataset emphasized three attack scenarios that could compromise in-vehicle functions rapidly and seriously or exacerbate the severity of an attack: flooding, fuzzy, and malfunction. To support these attack outlines, the researchers created two distinct datasets. The first dataset consisted of typical driving data without attacks, whereas the second dataset contained unusual driving data during an attack. Specifically, the researchers generated the attack data by injecting attack packets every 20 seconds for five seconds for each of the three attack scenarios. As shared in Table 5, the dataset was collected from three distinct cars: Hyundai YF Sonata, Kia Soul, and Chevrolet Spark.

- **Flooding Attack:** When an ECU node maintains a dominant status and seizes many resources assigned to the CAN bus, it is known as a flooding attack. This attack has the potential to disrupt regular driving and communication between ECU nodes. To execute the flooding attack, many payloads with a CAN ID of 0x000 were injected.
- **Fuzzy Attack:** In a fuzzy attack, the intruder infuses random CAN packets into the vehicle network repeatedly without any specific targets. The attack payloads were infused into the vehicle every 0.0003 seconds, and both the ID and data fields were subject to iterative injection during this process. The CAN IDs used were randomly generated and ranged from 0x000 to 0x7FF.
- **Malfunction Attack:** A malfunction attack intends to target a particular CAN ID from a particular vehicle. The researchers chose CAN IDs 0x316, 0x153, and 0x18E from the HYUNDAI YF Sonata, KIA Soul, and Chevrolet Spark cars respectively. Manipulating the data field of the selected CAN ID requires randomly injecting other CAN IDs as well. When the 8-byte data field values were changed to either 00 or a casual value, the vehicles showed abnormal responses.

Description of deep learning models

DL models have evolved as powerful tools in the field of artificial intelligence, enabling machines to learn and make predictions from complex data. These models are designed to mimic the formation and functioning of the human brain, with multiple layers of interlinked artificial neurons. By leveraging these layered architectures, deep learning models can automatically extract hierarchical representations of data, allowing for more abstract and nuanced understanding[44].

Compared to traditional rule-based or anomaly detection methods, deep learning models like LSTM, Bi-LSTM, GRU, and VGG-16 offer greater adaptability and effectiveness in detecting complex CAN bus attack patterns. Rule-based approaches rely on predefined signatures and thresholds, making them effective for known attack patterns but ineffective against novel or adaptive threats[2]. Anomaly detection methods, such as statistical or machine-learning-based techniques, can detect deviations from normal behavior but may struggle with high false-positive rates and require extensive feature engineering[45]. In contrast, deep learning models automatically learn temporal and spatial dependencies in CAN bus data, capturing subtle attack patterns without the need for handcrafted features[46,47].

The following sub-sections describe the working of the mentioned RNN variants and VGG-16.

Long short term memory (LSTM)

The introduction of LSTM was intended to consider the issue of vanishing and exploding gradients in RNNs[48]. This allowed the model to maintain its ability to handle short time lags, while also being able to bridge time intervals exceeding 1000 steps. Additionally, LSTM's proficiency in processing time series data and predicting sequences makes it a suitable option for examining the type of data produced by the CAN bus[49].

By utilizing the architecture of recurrent neural networks (RNN), it is possible to handle a data sequence x_1 ,...x_n, and through the application of RNN, a series of results y_1,...y_i can be generated[33].

$$h_t = f_W(h_{t-1}, x_t) \tag{1}$$

h_t = New State
f_W = Function with parameter W
h_{t-1} = Previous State
x_t = Input vector at timestamp t

Attack Name	Sonata	Soul	Spark
Flooding Attack	149,547	181,901	120,570
Fuzzy Attack	135,670	249,990	65,665
Malfunction Attack	132,651	173,436	79,787
Attack free State	117,173	192,516	136,934

Table 5. Overview of survival analysis dataset.

The mathematical formulations for the sigma cell of the typical recurrent network are presented below in (2), (3)

$$h_t = \sigma(W_h h_{t-1} + W_x x_t + \beta) \tag{2}$$

$$y_t = h_t \tag{3}$$

x_t = Input
h_t = Recurrent information
y_t = Cell's output at timestamp t
W_h and W_x = Weights
β = bias

The standard recurrent cells in recurrent networks have limitations in processing long-term dependencies. Learning the connection information becomes difficult as the disparity between associated inputs increases. The LSTM cell was introduced to address this issue, which incorporates a gate mechanism that enables the recurrent cell to store information. The LSTM cell is generally represented with a forget gate. The protection and control of the cell phase in LSTM are handled by three gates. Following Eqs. (4), (5), (6), (7), (8), and (9) define the input vector x_t, hidden vector h_{t-1} and output vector h_t.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + \beta_i) \tag{4}$$

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + \beta_f) \tag{5}$$

$$\vec{C}_t = \tanh(W_c[h_{t-1}, x_t] + \beta_c) \tag{6}$$

$$C_t = f_t * C_{t-1} + i_t * \vec{C}_t \tag{7}$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \tag{8}$$

$$h_t = o_t * \tanh(C_t) \tag{9}$$

W_f, W_i, W_c = Weights
$\beta_f, \beta_i, \beta_c$ = bias
C_t = New cell state
C_{t-1} = Previous cell state

The architecture of LSTM for classification purpose is shown in Fig. 3. In supervised learning, we have input variables (X) as well as an output variable (Y), and the mapping function $Y = f(X)$ that connects the input and output. The objective is to learn this mapping function precisely so that we can make predictions about the output variable (Y) for new input data (X). In our experimentation, we use various hyperparameters (detailed in Table 6) like, optimizer, learning rate, number of units, activation function, etc. A systematic investigation is done to decide the ideal hyperparameter values that yield the highest detection accuracy for our experiment. To train our LSTM classifier, we utilize a dataset consisting of benign and attack samples. We allocate 80% of the dataset for classifier training and 20% is used for testing. We assess the classifier's ability to detect attack instances.

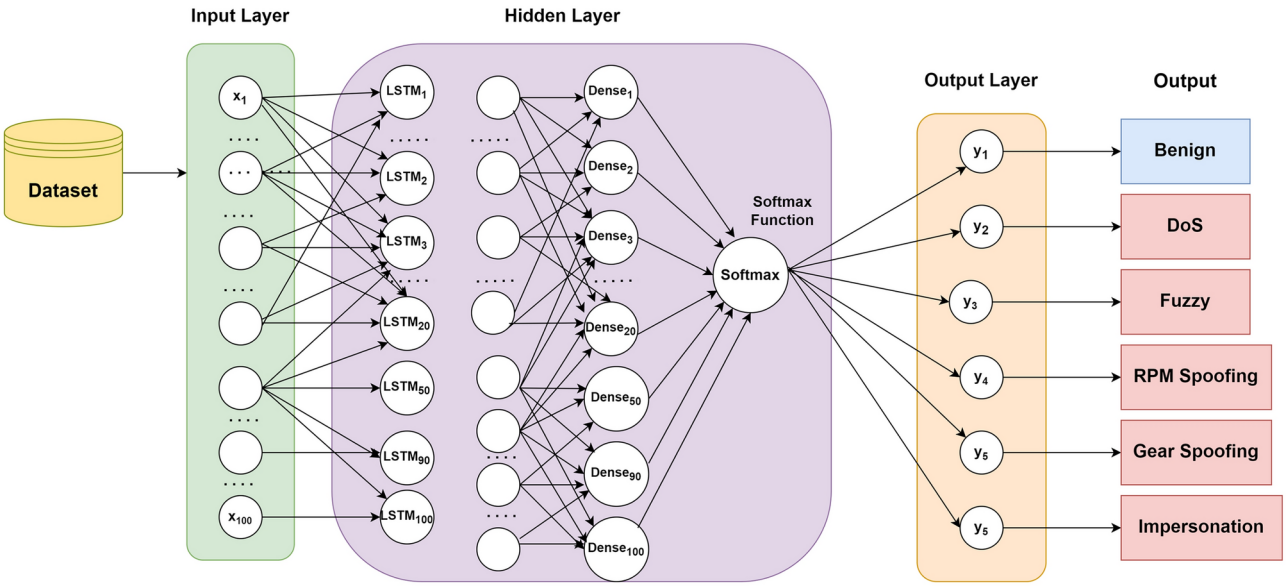

Fig. 3. Architecture of LSTM Model.

Dataset	Attributes	Value
Binary	Activation Function Input	relu
	Activation Function Output	sigmoid
	Optimizer	Adam
	Loss Function	binary crossentropy
	Epoch	100
	Batch size	512
Multiclass	Activation Function Input	relu
	Activation Function Output	softmax
	Optimizer	Adam
	Loss Function	categorical crossentropy
	Epoch	100
	Batch size	512

Table 6. LSTM attributes for binary and multiclass classification.

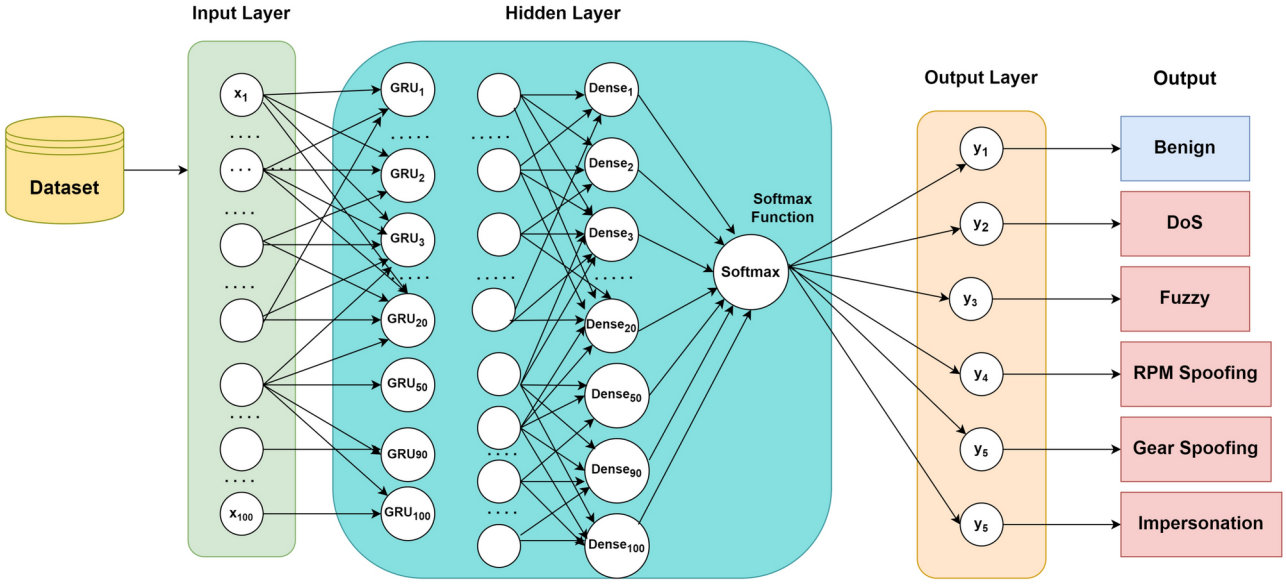

Fig. 4. Architecture of GRU model.

Gated recurrent unit (GRU)

GRU, shown in Fig. 4, is a form of RNN designed to tackle problems like long-term memory and gradients in backpropagation, similar to LSTM. It is an alternative to LSTM that aggregates the forget gate and input gate into a single update gate. Additionally, it consolidates the cell state and hidden state, as well as other adjustments, resulting in a more straightforward model than the standard LSTM. The observational findings in the literature suggest that LSTM and GRU have comparable proficiency on datasets with time sequence association, with GRU models even outperforming LSTM in some cases[34]. Initial research, as shown in Table 7 on GRU indicates that it has fewer parameters than LSTM, which implies that both training and inference time can be shortened. Moreover, studies demonstrate that GRU requires less time for both training and validation than LSTM across different datasets[50,51].

The internal structure of GRU is given as[34]:

h_{t-1} = State of previous timestamp t

x_t = Input at current timestamp t

h_t = Output at current timestamp t

r_t = Reset gate

z_t = Update gate

$\vec{h}_t$ = Output vector after processing the reset gate

The neural network's activation function incorporates the sigmoid function to restrict the gate's output to a range of 0 to 1.

The structure of GRU is explained below mathematically in Eqs. (10), (11), (12), (13),and (14):

Dataset	Parameters	Value
Binary	Activation Function Input	relu
	Activation Function Output	sigmoid
	Optimizer	Adam
	Loss Function	binary crossentropy
	Epoch	100
	Batch size	128
Multiclass	Activation Function Input	relu
	Activation Function Output	softmax
	Optimizer	Adam
	Loss Function	categorical cross-entropy
	Epoch	100
	Batch size	512

Table 7. GRU Attributes for binary and multiclass classification.

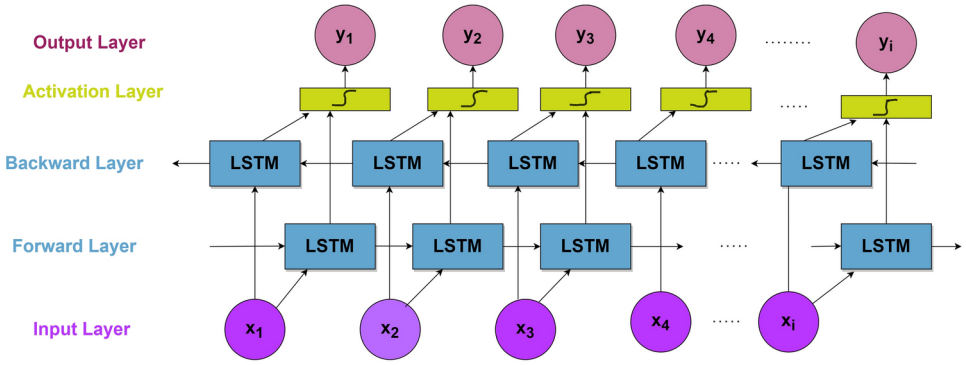

Fig. 5. Bi-directional LSTM model architecture.

$$S(x) = \frac{1}{1+e^{-x}} \tag{10}$$

$$r_t = \sigma(W_{rh}h_{t-1} + W_{rx}x_t) \tag{11}$$

$$z_t = \sigma(W_{zh}h_{t-1} + W_{zx}x_t) \tag{12}$$

$$\vec{h}_t = \tanh[W_{hh}(r_t * h_{t-1}) + W_{hx}x_t] \tag{13}$$

$$h_t = (1 - z_t) * \vec{h} + z_t * h_{t-1} \tag{14}$$

W_{rh} and W_{rx} = Reset gate parameters
W_{zh} and W_{zx} = Update gate parameters
W_{hh} and W_{hx} = Parameters utilized in acquiring the output vector $\vec{h}_t$
We employed GRU for both supervised binary classification as well as for supervised multiclass classification tasks.

Bi-directional LSTM

A Bi-directional LSTM is a kind of RNN model that consists of two LSTM layers working in opposite directions. It is designed to acquire details from both past and future contexts, making it particularly useful in sequence modeling tasks where the context in both directions is important[52]. As demonstrated in Fig. 5, in a bi-directional LSTM, the input sequence is processed in reverse, starting from the last element and moving toward the first element. This enables the network to acquire data from both past and future timestamps, giving a finer understanding of the sequence[53].

Bi-LSTM offers significant advantages over standard LSTM and GRU models in analyzing CAN bus messages for intrusion detection. Unlike unidirectional LSTM and GRU, Bi-LSTM processes data in both forward and backward directions, allowing it to capture dependencies from both past and future messages. This bidirectional approach enhances the model's ability to recognize complex attack patterns that unfold over multiple message intervals. Additionally, Bi-LSTM improves the detection of long-term dependencies, making it more effective in identifying anomalies within CAN bus traffic[54,55].

Dataset	Attributes	Value
Binary	Activation Function Input	relu
	Activation Function Output	sigmoid
	Optimizer	rmsprop
	Loss Function	binary crossentropy
	Epoch	100
	Batch size	128
Multiclass	Activation Function Input	tanh
	Activation Function Output	softmax
	Optimizer	Adam
	Loss Function	categorical cross-entropy
	Epoch	100
	Batch size	512

Table 8. Bi-LSTM attributes for binary and multiclass classification.

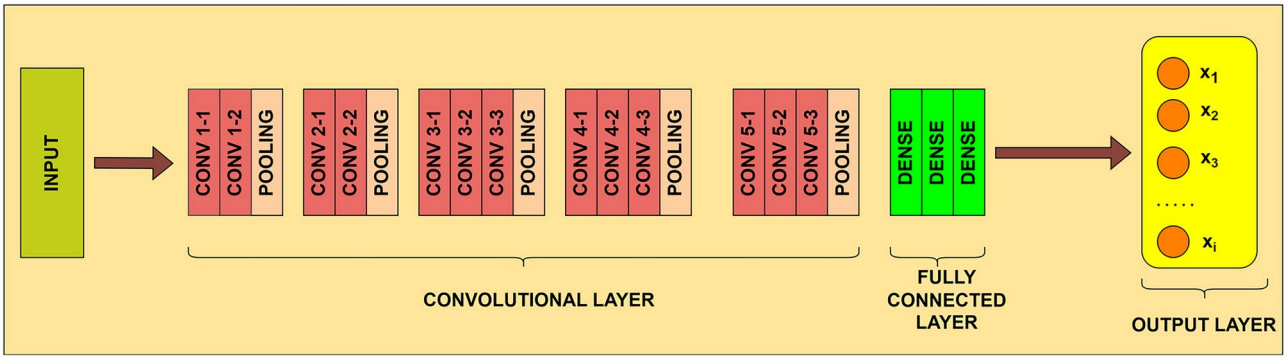

Fig. 6. VGG-16 model.

The architecture (as shown in Table 8) of a bi-directional LSTM typically has two LSTM layers: a frontward LSTM layer and a backward LSTM layer. The frontward LSTM layer handles it in the forward direction, whereas the backward LSTM layer processes it in the backward direction. Each LSTM layer sustains its own set of hidden states and cell states[56]. During training, the input sequence is presented to both the frontward and backward LSTM layers simultaneously. The frontward layer handles the sequence from the first element to the last, while the backward layer processes it from the last element to the first. The outputs of both layers are then combined in some way, such as concatenation or summation, to form the final output representation of each time step[57].

VGG-16

Converting sequential CAN bus data into images enables the application of CNNs. This transformation allows for enhanced pattern recognition, as CNNs can detect spatial relationships that traditional time-series models may struggle to capture. Additionally, CNNs automatically extract hierarchical features from image representations, improving intrusion detection performance. Visualizing attack patterns as images makes it easier to differentiate between normal and malicious behavior, aiding in anomaly detection[3]. Furthermore, this approach allows the use of pretrained models like VGG-16, which have been trained on large datasets such as ImageNet and can be fine-tuned for CAN bus anomaly detection. VGG-16 is particularly beneficial for intrusion detection in CAN bus systems due to its deep feature extraction capabilities, capturing intricate spatial relationships within transformed CAN data. Leveraging pretrained weights from ImageNet facilitates transfer learning, enabling the model to perform effectively. Its deep architecture ensures robustness against variations, allowing it to identify both common and rare attack patterns. Moreover, VGG-16 has demonstrated proven performance in various IDS applications, making it a reliable choice for detecting subtle anomalies in CAN bus data[58–60].

The VGG16 architecture is characterized by its depth and simplicity. As shown in Fig. 6, it comprises 16 convolutional and fully connected layers, organized sequentially. The key feature of VGG16 is its use of smaller 3x3 convolutional filters, which are stacked together multiple times. This enables the model to learn increasingly complex and abstract image representations by capturing local and global features. The VGG16 architecture is composed of five groups of convolutional layers, accompanied by a max pooling layer for spatial downsampling. These convolutional layers are accountable for retrieving hierarchical characteristics from the input image. The use of multiple convolutional layers in each group enables the network to learn a rich hierarchy of features at different levels of abstraction[61].

After the convolutional layers, VGG16 incorporates three fully connected layers, which serve as a classifier for the learned features. These fully connected layers gradually reduce the spatial dimensions of the features and make the final predictions based on the learned representations. The last fully connected layer typically has softmax activation, providing probability scores for different classes in a multi-class classification task[62].

Training VGG16 involves utilizing labeled image datasets for supervised learning. The network learns to classify images into different classes by adjusting its internal weights during the training process. Backpropagation is utilized to make this adjustment[63].

The complexity and depth of the VGG16 architecture make it computationally demanding, especially during training. However, its effectiveness in capturing rich image representations and achieving state-of-the-art performance on various benchmarks has established it as a widely used CNN architecture in the computer vision community[64,65]. The attributes of VGG-16 and its values utilized for the proposed work is shown in Table 9.

Methodology

This section describes the system and threat model by leveraging the OBD-II port and the proposed approach for intrusion detection.

System model

A CAN bus attack model involves targeting the communication between ECUs (Electronic Control Units) in systems like vehicles. The key components include:

- **CAN bus system**:

– Nodes: ECUs (e.g., engine, brakes, sensors).
– CAN Bus: Shared communication network.
– Messages: Data exchanged, prioritized for real-time needs.

- **Attack components**:

– Attacker Node: Device or compromised ECU injecting malicious messages.
– Malicious Actions: Message Injection, Flooding, and Spoofing.

Threat model

The CAN bus lacks authentication and encryption, making it susceptible to unauthorized message injection, interception, and manipulation. These vulnerabilities pose significant risks to vehicle safety, operational reliability, and data integrity. In this model, we define a range of attacks targeting the CAN bus through exploitation of the OBD-II port, a common access point for vehicle diagnostics. The OBD-II port allows attackers to monitor CAN traffic, identify critical CAN IDs, and inject malicious messages into the network. The following outlines key attack vectors:

1. **Denial of Service (DoS) Attack**: Attackers flood the CAN bus with high-priority messages, overwhelming the network and disrupting normal communication. Critical systems experience delays or loss of communication, potentially leading to failure of safety-critical functions.
2. **Fuzzy Attack**: Attackers inject random or malformed data into the CAN bus to exploit vulnerabilities in ECUs (Electronic Control Units), causing erratic and unpredictable system behavior. Malfunctions in ECUs may lead to inconsistent or dangerous behavior, such as incorrect sensor readings or actuator failures.
3. **Spoofing Attack**: Attackers manipulate CAN messages to impersonate legitimate ECUs, altering vehicle functions such as RPM or gear selection. Manipulated messages can affect vehicle performance and safety by causing unintended changes to critical systems.
4. **Impersonation Attack**: A more advanced form of spoofing, where attackers assume the identity of specific ECUs to gain unauthorized control over critical systems (e.g., brakes, engine control). Full control over vital vehicle functions, compromising safety and operational integrity.
5. **Malfunction Attack**: Attackers inject specifically crafted messages designed to trigger faults in ECUs, causing them to malfunction or fail. Critical systems may misbehave or shut down, leading to dangerous situations such as engine failure or loss of braking capability.

Attributes	Values
Activation Function Input	relu
Activation Function Output	softmax
Loss	Categorical Crossentropy
Optimizer	Adam
Epoch	100
Batch size	128

Table 9. VGG-16 attributes.

Proposed approach

This section outlines the proposed work's approach, which includes data preprocessing, data labeling, data merging, training, and testing. Figure 7 depicts the working of the proposed work.

1. **Data preprocessing**

(a) The column names were renamed to Timestamp, CAN ID, DLC, D[0], D[1], D[2], D[3], D[4], D[5], D[6], D[7], and Class to maintain uniformity across all datasets.
(b) Conversion of hexadecimal values to decimal values were performed for CAN ID and Data fields (D[0]-D[7]) to facilitate numerical processing by DL models.
(c) Some of the CAN IDs were in exponential form (e.g., 5.00E+04) and were converted into their correct decimal representations before further processing.
(d) Empty data fields were replaced with zero to maintain data consistency.
(e) Random undersampling was applied to balance the minor class data, ensuring an equal representation of different attack and normal data points.
(f) CAN message data was extracted and reshaped into a 9×9×3 array by grouping consecutive rows to form an image patch. The images are created in RGB format.
(g) The reshaped array was converted into an image format using *Image.fromarray()* and saved as a PNG file.
(h) The generated image patches were then resized to 224×224 pixels to fit the VGG16 input requirements.

2. **Feature extraction**

(a) Sequence-Based Feature Extraction: We have utilized Timestamp, CAN ID, and Data fields (D[0]-D[7]) from the CAN bus data which capture the temporal dependencies in CAN messages. The RNN models utilize these features.
(b) Image-Based Feature Extraction: The CAN bus data is transformed into image representations to leverage spatial feature extraction. The VGG-16 model is employed as a feature extractor. Low-level features such as edges, and textures are extracted from the initial convolutional layers, while high-level features like structural patterns and attack signatures are extracted from pooling layers.

3. **Data labelling**

(a) For binary class data, labels were attack and benign.
(b) For multiclass data, labels were Benign, DoS, Fuzzy, RPM Spoofing, Gear Spoofing, and Impersonation.
(c) We assigned an integer number to each label to maintain the uniformity of the dataset (Table 10).

4. **Dataset merging**

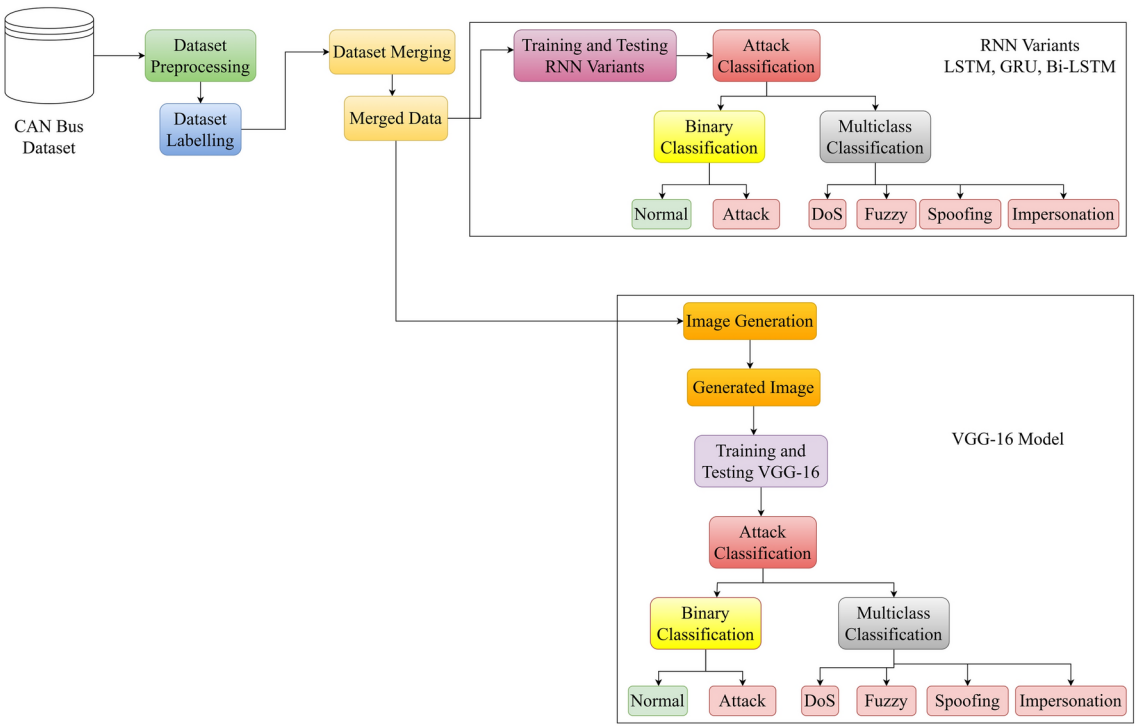

Fig. 7. Flowchart of Proposed IDS.

Attacks Types	Labels	Number
Benign	R	0
DoS Attack	T1	1
Fuzzy Attack	T2	2
RPM Spoofing Attack	T3	3
Gear Spoofing Attack	T4	4
Impersonation Attack	T5	5

Table 10. Tabels for labelling of samples.

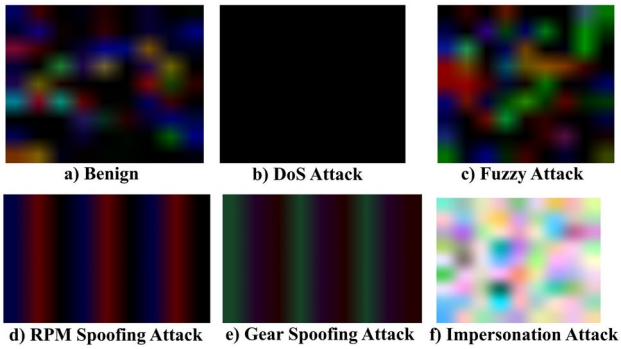

Fig. 8. Images generated by VGG-16 architecture.

(a) Timestamp column was converted from UNIX format to UTC format to ensure that the samples in the merged dataset can be easily sorted based on their timestamp.
(b) The timestamp values were converted to a UTC datetime format using the datetime module in Python.
(c) We obtained a combined dataset with a size of (8184143 X 11) in CSV format.
(d) The merged datasets were used to create images for the VGG-16 model as shown in Fig. 8.

5. **Training and Testing**

(a) The merged dataset consisted of 6,867,882 benign messages and 1,316,261 attack messages.
(b) For multiclass classification, we categorized the attack messages into 6 different classes.
(c) In addition to the message data, we also created image data for the VGG16 model and categorized images into 6 classes.
(d) We divide the dataset into training and testing sets using an 80% - 20% scale. The architectures were then trained on the training set and tested on the testing set.

While all three datasets come from different experiments and have non-overlapping timestamps, the merging of these datasets serves a critical role in the analysis presented in the paper. Each data set brings unique insights from different experimental conditions or data collection processes and combining them provides a more comprehensive and robust data set for further analysis. The necessity of merging lies in creating a more holistic representation of the phenomenon being studied. For the spatial features in the dataset, we have generated images from the CAN bus dataset for each type of label (normal and attack). Each pixel in the image represents encoded CAN bus data (e.g., CAN ID, payload values). Differences in pixel intensities reflect variations in normal and attack patterns. For the temporal features, we have utilized the sequence of CAN bus data for normal and attack data. Normal sequences have a structured order of CAN messages. Attack data disrupts this order by injecting unexpected messages

By merging datasets, we increase the diversity of the data, which is particularly important for training DL models that need to generalize well across different conditions. This combined dataset is more representative of real-world scenarios, where data is often collected from various sources, each with its characteristics. Also, while the datasets are from different sources, the preprocessing and harmonization techniques applied during the merging process ensure that the data becomes compatible, filling gaps and resolving discrepancies in format or missing values. The contribution of this paper lies not only in merging the data but also in the careful preprocessing that enables a more reliable and unified dataset for analysis.

Experimental setup and results

Below, we discuss the experiment setup and the results generated in detail:

Experimental setup

The applied technique is put into action, and its results are assessed with the following specifications: an Intel(R) Core(TM) i5-8265U CPU @ 1.60GHz 1.80 GHz processor and 8 GB of RAM. The suggested strategy is executed and simulated within a Python-based setting, employing the Keras 2.12.0 backend through Google Colab.

Performance measures

We have employed accuracy, precision, recall, and F1 scores as assessment measures. These measures are defined below:

1. **Accuracy:** It is characterized as the proportion of accurately detected instances compared to the overall number of detections.

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN} \tag{15}$$

2. **Precision:** It represents the relationship between accurately identified instances of misbehavior and the total number of misbehavior detections. A lower precision value suggests a higher occurrence of false positives within the model.

$$Precision = \frac{TP}{TP + FP} \tag{16}$$

3. **Recall:** It is the proportion of accurately detected instances of misbehavior compared to the total number of real misbehavior occurrences. A lower recall value shows the model's inability to properly detect misbehavior.

$$Recall = \frac{TP}{TP + FN} \tag{17}$$

4. **F1 Score:** It is the harmonic mean between precision and recall. It quantifies the overall detection effectiveness of an architecture.

$$F1\ Score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{18}$$

TP (True Positive): The model correctly predicts a positive class.
FP (False Positive): The model incorrectly predicts a positive class for a negative instance.
TN (True Negative): The model correctly predicts a negative class.
FN (False Negative): The model incorrectly predicts a negative class for a positive instance.

Results

The outcomes obtained from employing deep learning models on various datasets for binary and multiclass classification are presented in this section.

For binary classification (Table 11), the results indicate that LSTM consistently achieves the highest accuracy on individual datasets, particularly excelling in the Car Hacking (0.9989) and Survival (0.9987) datasets, with high precision, recall, and F1-scores. However, its performance deteriorates on the Combined dataset, where it records a notable drop in recall (0.2547) and F1-score (0.4058), suggesting challenges in handling diverse attack types. GRU, while performing reasonably well on individual datasets, exhibits lower precision and recall, particularly on the OTIDS and Combined datasets, indicating limitations in its generalization capabilities.

Bidirectional LSTM offers a balance between recall and precision across datasets, though it struggles with low precision (0.193) on the Combined dataset, resulting in an imbalanced performance. In contrast, VGG-16, despite being a CNN-based model, demonstrates strong generalization across datasets, achieving the highest accuracy (0.9462) on the Combined dataset and a well-balanced F1-score (0.923). The loss values further reinforce these findings, with LSTM maintaining the lowest loss on individual datasets but experiencing an increase in the Combined dataset. GRU exhibits the highest loss (0.081) on the Survival dataset, indicating potential instability in certain scenarios. Overall, the results suggest that while LSTM is highly effective for specific attack detection in individual datasets, its performance is compromised when handling diverse attack patterns in a combined setting. VGG-16 emerges as a robust alternative, demonstrating stable performance across all datasets. Bidirectional LSTM remains a viable option, though its inconsistencies in precision needs further optimization.

In the case of multiclass labels, LSTM has achieved an accuracy of 100% for both OTIDS and survival analysis datasets in the benign label. LSTM models can capture complex patterns and dependencies in the data, enabling them to learn intricate associations across multiple classes. They can model non-linear relationships, making them effective in multiclass classification tasks where the decision boundaries between classes may be complex and require capturing higher-level patterns and dependencies. Among the given models, VGG-16 has demonstrated better performance. The VGG-16 model includes a deep CNN design with 16 layers, allowing it to learn complicated image characteristics and patterns. This depth enables the model to capture hierarchical

Model	Metric	Car Hacking	OTIDS	Survival	Combined
LSTM	Accuracy	0.9989	0.9807	0.9987	0.8801
	Precision	0.9997	1	0.9955	1
	Recall	0.9948	0.9923	0.9923	0.2547
	F1-Score	0.9972	0.9939	0.9939	0.4058
	Loss	0.003	0.03	0.005	0.019
GRU	Accuracy	0.9932	0.9288	0.9092	0.8802
	Precision	0.7325	0.7944	0.8912	1
	Recall	0.8527	0.8507	0.9525	0.255
	F1-Score	0.788	0.8216	0.9208	0.4064
	Loss	0.0015	0.03	0.081	0.019
Bi-LSTM	Accuracy	0.8122	0.9337	0.9351	0.8851
	Precision	0.9324	0.73583	0.9927	0.193
	Recall	0.8547	0.8511	0.8473	0.9984
	F1-Score	0.8919	0.789	0.9143	0.9234
	Loss	0.0015	0.03	0.0013	0.011
VGG16	Accuracy	0.8974	0.8995	0.9102	0.9462
	Precision	0.9437	0.9437	0.9427	0.9352
	Recall	0.8975	0.8995	0.8989	0.9121
	F1-Score	0.9071	0.9086	0.908	0.923
	Loss	0.010	0.010	0.089	0.053

Table 11. Deep learning results on individual datasets for binary labels.

Model	Metric	Car Hacking				
		DoS	Fuzzy	RPM Spoof	Gear Spoof	Benign
LSTM	Accuracy	0.99	0.99	0.85	0.82	0.99
	Precision	0.81	0.99	0.79	0.72	0.81
	Recall	0.83	0.85	0.82	0.88	1
	F1-Score	0.92	0.91	0.80	0.79	0.90
GRU	Accuracy	0.81	0.83	0.79	0.73	0.88
	Precision	0.81	0.96	0.79	0.74	0.81
	Recall	0.83	0.85	0.82	0.88	1
	F1-Score	0.82	0.91	0.80	0.79	0.90
Bi-LSTM	Accuracy	0.91	0.82	0.76	0.79	0.93
	Precision	0.85	0.84	0.84	0.79	0.79
	Recall	0.83	0.85	0.65	0.65	1
	F1-Score	0.84	0.85	0.71	0.71	0.88
VGG16	Accuracy	1	0.99	1	0.96	0.96
	Precision	1	0.94	1	1	1
	Recall	1	0.99	1	0.96	0.98
	F1-Score	1	0.90	1	0.98	0.97

Table 12. Deep learning results on car hacking dataset for multiclass labels.

representations of the input data, which is helpful for multiclass classification tasks with complex visual patterns. VGG-16 has achieved an accuracy of 100% for the car hacking dataset for the DoS and RPM spoofing attack, the OTIDS dataset regarding the DoS and impersonation attack, and the survival analysis dataset regarding flooding and malfunction attack. For the precision score, VGG-16 (Table 12) has achieved a precision of 100% for the car hacking dataset in terms of DoS, RPM spoofing, gear spoofing, and benign label.

For the OTIDS dataset, VGG-16 (Table 13) has best performed for DoS and impersonation attacks with a precision score of 100%. The survival analysis dataset has achieved a precision score of 100% for flooding and malfunction attacks. For recall, LSTM, GRU, and VGG-16 have achieved a recall score of 1 for benign data. For DoS and RPM attacks, VGG-16 has earned a recall of 100%.

For the Survival Analysis dataset, VGG-16 has gained a recall of 100% for DoS and impersonation attacks. LSTM, GRU, and bi-LSTM have achieved a recall of 100% for benign data. For the survival dataset (Table 14), a recall of 100% is conducted by VGG-16 for flooding and malfunction attacks. For the F1-score measure, VGG-16 has performed an F1-score of 100% for the car hacking dataset in the case of DoS and RPM spoofing intrusions.

Model	Metric	OTIDS			
		DoS	Fuzzy	Impersonation	Benign
LSTM	Accuracy	0.95	0.81	0.82	1
	Precision	0.81	0.82	0.85	0.99
	Recall	0.82	0.81	0.82	1
	F1-Score	1	0.82	0.84	0.99
GRU	Accuracy	0.91	0.81	0.82	0.93
	Precision	0.89	0.82	0.81	0.99
	Recall	0.89	0.81	0.89	1
	F1-Score	0.89	0.84	0.84	0.99
Bi-LSTM	Accuracy	0.85	0.79	0.83	0.99
	Precision	0.89	0.82	0.84	0.99
	Recall	0.89	0.81	0.77	1
	F1-Score	0.89	0.82	0.82	0.99
VGG-16	Accuracy	1	0.98	1	0.98
	Precision	1	0.94	1	1
	Recall	1	0.99	1	0.98
	F1-Score	1	0.90	1	0.98

Table 13. Deep learning results on OTIDS dataset for multiclass labels.

Model	Metric	Survival			
		Flooding	Fuzzy	Malfunction	Benign
LSTM	Accuracy	0.99	0.91	0.89	1
	Precision	0.91	0.88	0.88	0.95
	Recall	0.99	0.72	0.77	0.99
	F1-Score	0.96	0.79	0.82	0.96
GRU	Accuracy	0.95	0.76	0.72	0.93
	Precision	0.92	0.75	0.84	0.94
	Recall	0.95	0.72	0.77	0.99
	F1-Score	0.93	0.79	0.82	0.96
Bi-LSTM	Accuracy	0.85	0.73	0.80	0.95
	Precision	0.89	0.75	0.81	0.94
	Recall	0.88	0.72	0.77	0.99
	F1-Score	0.88	0.73	0.78	0.96
VGG16	Accuracy	1	0.99	1	0.78
	Precision	1	0.54	1	0.99
	Recall	1	0.99	1	0.78
	F1-Score	1	0.698	1	0.87

Table 14. Deep learning results on survival analysis dataset for multiclass labels.

In the case of the DoS attack in the OTIDS dataset, both LSTM and VGG-16 have gained an F1-score of 100%. In the case of an impersonation attack, an F1-score of 100% is achieved by VGG-16. VGG-16 has performed an F1-score of 100% for the survival analysis dataset for flooding and malfunction attacks.

In the case of the combined dataset (Table 15), the VGG-16 has outperformed the mentioned other DL models for binary and multiclass labels. The model has achieved the highest accuracy, precision, recall, and F1-score of 100% in the case of a DoS attack for multiclass labels. Based on the results acquired, it can be inferred that enhancing performance on the consolidated dataset involves integrating both the temporal and spatial characteristics of the dataset.

Comparison with machine learning models

To demonstrate the superiority of DL models over traditional ML approaches, we conducted a comparative analysis between our proposed method and previous works. The comparison has been presented on two datasets. Among the four DL models evaluated, VGG-16 exhibited the best performance; therefore, we have focused our comparison on the results obtained using the VGG-16 model. The comparative analysis of intrusion detection approaches on the Car Hacking Dataset is presented in Table 16. Anjum et al.[66] employed XGBoost, achieving an accuracy of over 99% for all attack types, with the highest F1-score of 0.9963 for Fuzzy attacks and the lowest

Model	Metric	Combined			
		DoS	Impersonation	Malfunction	Benign
LSTM	Accuracy	0.92	1	0.05	1
	Precision	0.83	1	0.07	0.99
	Recall	0.06	1	1	0.48
	F1-Score	0.11	1	0.14	0.65
GRU	Accuracy	0.82	1	0.08	1
	Precision	0.83	1	0.07	0.99
	Recall	0.06	1	1	0.48
	F1-Score	0.11	1	0.14	0.65
Bi-LSTM	Accuracy	0.82	1	0.02	0.99
	Precision	0.83	1	0.07	0.99
	Recall	0.06	1	1	0.52
	F1-Score	0.11	1	0.52	0.67
VGG16	Accuracy	1	1	1	0.90
	Precision	1	1	1	1
	Recall	1	0.96	0.96	0.89
	F1-Score	1	0.98	0.98	0.94

Table 15. Deep learning results on combined dataset for multiclass labels

Reference	Approach	Metric	DoS	Fuzzy	RPM Spoofing	Gear Spoofing
Anjum et al.[66]	XGBoost	Accuracy	0.9976	0.9990	0.9858	0.9864
		Precision	0.9852	0.9999	1	0.9982
		Recall	1	0.9926	0.90	0.9006
		F1-score	0.9926	0.9963	0.9474	0.9469
Refat et al.[67]	SVM	Accuracy	0.9990	0.9943	0.9643	-
		Precision	0.994	0.996	0.9761	-
		Recall	0.9969	0.9952	0.9363	-
		F1-score	0.9981	0.9973	0.9537	-
Ours	VGG-16	Accuracy	1	0.99	1	0.96
		Precision	1	0.94	1	1
		Recall	1	0.99	1	0.96
		F1-score	1	0.90	1	0.98

Table 16. Comparison on car hacking dataset.

of 0.9469 for Gear Spoofing. Refat et al.[67] utilized SVM, obtaining comparable accuracy, though Gear Spoofing detection was not evaluated. Their model achieved an F1-score of 0.9973 for Fuzzy attacks but exhibited a lower value of 0.9537 for RPM Spoofing. The proposed VGG-16 model outperforms existing methods in detecting DoS and RPM Spoofing attacks with perfect scores (accuracy, precision, recall, and F1-score = 1.00). For Fuzzy attacks, the VGG-16 model shows slightly lower performance with an F1-score of 0.90, whereas its Gear Spoofing detection achieves an accuracy of 0.96 and an F1-score of 0.98. Overall, the VGG-16 model demonstrates robust performance, particularly excelling in DoS and RPM spoofing detection, thereby affirming its effectiveness in automotive intrusion detection.

The comparative evaluation of different intrusion detection approaches on the OTIDS dataset is shown in Table 17. Moulahi et al.[68] employed a Decision Tree classifier, which demonstrated relatively low effectiveness, particularly in detecting fuzzy attack, with a precision of 0.233 and an F1-score of 0.232. The model performed better in identifying impersonation attack, achieving an F1-score of 0.990. Bari et al.[35] used SVM, which exhibited significantly improved performance compared to the DT approach, achieving an F1-score of 0.99 for DoS, 0.97 for Fuzzy, and 0.95 for impersonation attack. The proposed VGG-16 model outperforms previous methods, achieving perfect classification (accuracy, precision, recall, and F1-score of 1.00) for DoS and impersonation attacks. Overall, the results demonstrate the superiority of VGG-16 for intrusion detection.

Discussion

This research primarily aims to detect intrusion attacks within both singular and combined datasets. The goal is to examine the capability of DL models in recognizing a comprehensive range of attacks targeting in-vehicle networks. For this purpose, we utilized variants of RNN such as LSTM, GRU, and Bi-directional LSTM to learn

Reference	Approach	Metric	DoS	Fuzzy	Impersonation
Moulahi et al[68]	Decision Tree	Accuracy	-	-	-
		Precision	0.768	0.233	0.990
		Recall	0.757	0.230	0.990
		F1-score	0.762	0.232	0.990
Bari et al.[35]	SVM	Accuracy	-	-	-
		Precision	0.99	1	0.96
		Recall	0.99	0.96	0.94
		F1-score	0.99	0.97	0.95
Ours	VGG-16	Accuracy	1	0.98	1
		Precision	1	0.94	1
		Recall	1	0.99	1
		F1-score	1	0.90	1

Table 17. Comparison on OTIDS dataset.

the temporal features of the dataset. We have also employed VGG-16, a type of CNN, to understand spatial features. We tested the mentioned DL models for binary and multiclass labels.

LSTM and GRU, as recurrent models, excel in capturing temporal dependencies, making them highly effective for detecting DoS attacks in binary classification. However, their reliance on sequential processing leads to misclassifications in multiclass scenarios, where overlapping attack characteristics reduce their precision. VGG-16, with its convolutional feature extraction, maintains robust performance across both classification tasks, effectively identifying DoS attacks even when trained on multiple datasets. For fuzzy attacks, LSTM and GRU benefit from their ability to model long-term dependencies but struggle in multiclass settings due to feature similarities with normal traffic. Their sequential nature limits their ability to extract complex spatial patterns. In contrast, VGG-16 leverages deep feature learning to accurately differentiate fuzzy attacks from other anomalies, making it more reliable for distinguishing subtle attack variations. When detecting spoofing attacks, LSTM and GRU exhibit moderate performance, often misclassifying spoofing as other attack types due to insufficient spatial feature extraction. Bidirectional LSTM provides slight improvement by utilizing past and future context, but the enhancement remains limited. VGG-16, with its hierarchical convolutional layers, effectively distinguishes spoofing attacks by capturing fine-grained differences in attack patterns. Its ability to extract spatial and structural features makes it more suited for complex attack scenarios.

To prevent overfitting in the VGG-16 classification model, several techniques were applied. Data augmentation was performed using the ImageDataGenerator to rescale images, helping the model generalize better. A pre-trained VGG-16 model was used with transfer learning by freezing its convolutional layers to retain learned features while training only the new classifier layers. A GlobalAveragePooling2D layer was added to reduce parameters, followed by dense layers with dropout to minimize overfitting. The Adam optimizer with a learning rate of 0.001 was used for efficient training. Early stopping was implemented to halt training if validation accuracy stopped improving, preventing unnecessary overfitting. To avoid premature stopping due to minor fluctuations, a patience value is set. This determines the number of consecutive epochs the model is allowed to continue training without improvement before stopping. At each epoch, if the monitored validation metric improves, the model continues training. If no improvement is observed for the number of epochs specified by the patience parameter, training stops. Lastly, a checkpoint mechanism was used to save the best model based on validation accuracy.

The deep learning approach for real-time intrusion detection in CAN bus networks performs by rapidly identifying anomalies and cyberattacks with high accuracy. It continuously monitors CAN messages and learns patterns from normal traffic. When an intrusion occurs, such as, spoofing, or DoS attacks, the model detects deviations in timing, or frequency. VGG-16 transforms CAN data into images, allowing the network to recognize malicious patterns visually. On the other hand, LSTM and GRU analyze message sequences and detect irregularities based on learned normal behavior. Bi-LSTM improves anomaly detection by processing sequences bidirectionally. These approaches excel in low-latency detection, ensuring real-time protection without significantly impacting system performance. Optimized models can run on embedded automotive hardware, responding instantly to threats by flagging suspicious messages, triggering alerts, or blocking malicious traffic. Overall, it enhances vehicle cybersecurity by providing fast, adaptive, and efficient anomaly detection in dynamic automotive environments.

Since we have utilized three publicly available datasets to cover various attack types, our approach enhances the model's ability to learn diverse attack patterns from different vehicle models. By combining these datasets into a single file, we enable the DL models to recognize variations in attack behaviors across multiple sources. This approach improves generalization by exposing the model to a broader range of CAN bus attack signatures, reducing the risk of overfitting to a single dataset. We ensured a balanced representation of attack and normal data, addressing potential inconsistencies between datasets, and validating the model's performance on unseen attacks to increase robustness against novel and adaptive threats.

Recurrent Neural Networks (RNNs) are highly effective in capturing temporal dependencies in sequential data, making them well-suited for anomaly detection in time-series data such as CAN bus messages. However, their sequential nature leads to high latency, which can be a significant issue for real-time IDS applications in

vehicles. Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) networks mitigate vanishing gradient issues, making them more viable alternatives to standard RNNs. Nevertheless, their real-time deployment necessitates optimization techniques such as quantization and model pruning.

In contrast, CNNs excel in feature extraction and classification, offering faster processing compared to RNNs. Their ability to perform parallel computations makes them more hardware-friendly and suitable for real-time deployment in automotive embedded systems. Effective real-time IDS deployment requires lightweight models optimized for inference, such as pruned LSTMs, GRUs, or compact CNN architectures like MobileNet and TinyCNN[69]. For real-time decision-making, low-power microcontrollers or specialized processing units, such as Tensor Processing Units (TPUs) (e.g., Google Coral, NVIDIA Jetson), are preferred. However, for more computationally intensive tasks, higher processing power is necessary, typically requiring a dedicated AI accelerator or an FPGA-based system[70,71] to efficiently run CNN models.

Conclusion and future work

In this research, we evaluate various deep-learning techniques for mitigating security threats within in-vehicle systems. The method combines deep learning techniques with temporal and spatial representations of CAN bus data. Spatial features are utilized to represent network traffic as traffic images, while temporal characteristics capture the dependencies among features over time. These datasets are consolidated into a single file and employed to anticipate the category of each message. The objective is to identify all potential attacks that can impact the intra-vehicle network. The output of the proposed approach is evaluated, and it is concluded that in the case of the binary dataset, LSTM has performed better than other deep learning models. For the multiclass dataset, VGG-16 has achieved better results.

Although the proposed approach demonstrates promising results, it falls short of attaining perfect detection of all automotive cyber-attacks. Further investigation could explore potential enhancements to enhance the classification proficiency of the proposed approach by combining both spatial and temporal representations of the vehicle data. The authors intend to explore additional RNN variants and other deep learning architectures to further enhance detection performance.

DL-based automotive intrusion detection systems are generally more complex and resource-intensive than traditional or machine learning approaches. Given the limited computing and memory capacity of in-vehicle networks, deploying such models remains a significant challenge. Instead of integrating a deep learning IDS into existing ECUs, an alternative approach could be to introduce it as a separate node. Since many in-vehicle networks, particularly the CAN bus, operate on a broadcast mechanism, this IDS node would have access to all communications on the bus. By equipping the new node with dedicated hardware and software to support deep learning, the computational load would be offloaded from the IVN itself, ensuring efficient deployment without straining existing vehicle resources.

While DL models have shown promising results in intrusion detection, their deployment on resource-constrained automotive ECUs may introduce latency issues. The computational demands of these models, particularly for high-dimensional CAN bus data, necessitate optimization techniques such as model pruning, quantization, and knowledge distillation to reduce inference time and memory footprint. Additionally, real-time performance is crucial for ensuring timely detection and response to cyber threats in vehicular networks. Future work will focus on evaluating the trade-off between detection accuracy and computational efficiency by testing these models on embedded automotive platforms, assessing power consumption, and optimizing their architecture to meet real-time constraints without compromising detection performance.

Data availability

The datasets analyzed during the current study are available in the HCRL website https://ocslab.hksecurity.net. The merged preprocessed dataset can be accessed through https://github.com/Rrai997/CAN_Bus_IDS. The links to access the public datasets are mentioned below: Car Hacking Dataset: https://ocslab.hksecurity.net/Datasets/car-hacking-dataset Survival Analysis Dataset: https://ocslab.hksecurity.net/Datasets/survival-ids OTIDS Dataset: https://ocslab.hksecurity.net/Dataset/CAN-intrusion-dataset.

Received: 17 February 2025; Accepted: 11 April 2025
Published online: 22 April 2025

References

1. Wu, W. et al. A survey of intrusion detection for in-vehicle networks. *IEEE Transactions on Intelligent Transportation Systems* **21**, 919–933 (2019).
2. Rajapaksha, S. et al. Ai-based intrusion detection systems for in-vehicle networks: A survey. *ACM Computing Surveys* **55**, 1–40 (2023).
3. Javed, A. R., Ur Rehman, S., Khan, M. U., Alazab, M. & Reddy, T. Canintelliids: Detecting in-vehicle intrusion attacks on a controller area network using cnn and attention-based gru. *IEEE transactions on network science and engineering* **8**, 1456–1466 (2021).
4. Cheng, P., Xu, K., Li, S. & Han, M. Tcan-ids: intrusion detection system for internet of vehicle using temporal convolutional attention network. *Symmetry* **14**, 310 (2022).
5. Michaels, A. J. et al. Can bus message authentication via co-channel rf watermark. *IEEE Transactions on Vehicular Technology* **71**, 3670–3686 (2022).
6. Cheng, K. et al. Caneleon: Protecting can bus with frame id chameleon. *IEEE Transactions on Vehicular Technology* **69**, 7116–7130. https://doi.org/10.1109/TVT.2020.2990417 (2020).
7. Hu, R., Wu, Z., Xu, Y. & Lai, T. Multi-attack and multi-classification intrusion detection for vehicle-mounted networks based on mosaic-coded convolutional neural network. *Scientific Reports* **12**, 6295 (2022).
8. Lo, W. et al. A hybrid deep learning based intrusion detection system using spatial-temporal representation of in-vehicle network traffic. *Vehicular Communications* **35**, 100471 (2022).

9. Bozdal, M., Samie, M. & Jennions, I. K. Winds: A wavelet-based intrusion detection system for controller area network (can). *IEEE Access* **9**, 58621–58633 (2021).
10. Lin, C.-W. & Sangiovanni-Vincentelli, A. Cyber-security for the controller area network (can) communication protocol. In *2012 International Conference on Cyber Security*, 1–7 (IEEE, 2012).
11. Muthamil Sudar, K. & Deepalakshmi, P. An intelligent flow-based and signature-based ids for sdns using ensemble feature selection and a multi-layer machine learning-based classifier. *Journal of Intelligent & Fuzzy Systems* **40**, 4237–4256 (2021).
12. Heidari, A., Navimipour, N. J. & Unal, M. A secure intrusion detection platform using blockchain and radial basis function neural networks for internet of drones. *IEEE Internet of Things Journal* **10**, 8445–8454 (2023).
13. Young, C., Olufowobi, H., Bloom, G. & Zambreno, J. Automotive intrusion detection based on constant can message frequencies across vehicle driving modes. In *Proceedings of the ACM Workshop on Automotive Cybersecurity*, 9–14 (2019).
14. Xun, Y., Deng, Z., Liu, J. & Zhao, Y. Side channel analysis: A novel intrusion detection system based on vehicle voltage signals. *IEEE Transactions on Vehicular Technology* **72**, 7240–7250 (2023).
15. Rai, R. & Grover, J. Comparative analysis of cosine and jaccard similarity-based classification for detecting can bus attacks. In *2024 IEEE Region 10 Symposium (TENSYMP)*, 1–6 (IEEE, 2024).
16. Amiri, Z., Heidari, A. & Navimipour, N. J. Comprehensive survey of artificial intelligence techniques and strategies for climate change mitigation. *Energy* 132827 (2024).
17. Amiri, Z., Heidari, A., Jafari, N. & Hosseinzadeh, M. Deep study on autonomous learning techniques for complex pattern recognition in interconnected information systems. *Computer Science Review* **54**, 100666 (2024).
18. Qin, H., Yan, M. & Ji, H. Application of controller area network (can) bus anomaly detection based on time series prediction. *Vehicular Communications* **27**, 100291 (2021).
19. Mansourian, P., Zhang, N., Jaekel, A. & Kneppers, M. Deep learning-based anomaly detection for connected autonomous vehicles using spatiotemporal information. *IEEE Transactions on Intelligent Transportation Systems* **24**, 16006–16017 (2023).
20. Chen, A., He, Z. & Zhang, D. An anomaly detection model for can networks based on cnn and transformer. In *IECON 2024-50th Annual Conference of the IEEE Industrial Electronics Society*, 1–6 (IEEE, 2024).
21. Kishore, C. R., Rao, D. C., Nayak, J. & Behera, H. Intelligent intrusion detection framework for anomaly-based can bus network using bidirectional long short-term memory. *Journal of The Institution of Engineers (India): Series B* 1–24 (2024).
22. Al-Jarrah, O. Y., El Haloui, K., Dianati, M. & Maple, C. A novel detection approach of unknown cyber-attacks for intra-vehicle networks using recurrence plots and neural networks. *IEEE Open Journal of Vehicular Technology* **4**, 271–280 (2023).
23. Wang, K., Zhang, A., Sun, H. & Wang, B. Analysis of recent deep-learning-based intrusion detection methods for in-vehicle network. *IEEE Transactions on Intelligent Transportation Systems* **24**, 1843–1854 (2022).
24. Aliwa, E., Rana, O., Perera, C. & Burnap, P. Cyberattacks and countermeasures for in-vehicle networks. *ACM Computing Surveys (CSUR)* **54**, 1–37 (2021).
25. Song, H. M., Woo, J. & Kim, H. K. In-vehicle network intrusion detection using deep convolutional neural network. *Vehicular Communications* **21**, 100198 (2020).
26. Khan, J., Lim, D.-W. & Kim, Y.-S. Intrusion detection system can-bus in-vehicle networks based on the statistical characteristics of attacks. *Sensors* **23**, 3554 (2023).
27. Lee, H., Jeong, S. H. & Kim, H. K. Otids: A novel intrusion detection system for in-vehicle network by using remote frame. In *2017 15th Annual Conference on Privacy, Security and Trust (PST)*, 57–5709 (IEEE, 2017).
28. Young, C., Svoboda, J. & Zambreno, J. Towards reverse engineering controller area network messages using machine learning. In *2020 IEEE 6th World Forum on Internet of Things (WF-IoT)*, 1–6 (IEEE, 2020).
29. Han, M. L., Kwak, B. I. & Kim, H. K. Anomaly intrusion detection method for vehicular networks based on survival analysis. *Vehicular communications* **14**, 52–63 (2018).
30. Rai, R., Grover, J. & Pandey, S. Unveiling threats: A comprehensive taxonomy of attacks in in-vehicle networks. In *2023 16th International Conference on Security of Information and Networks (SIN)*, 1–7 (IEEE, 2023).
31. Sudar, K. M., Deepalakshmi, P., Singh, A. & Srinivasu, P. N. Tfad: Tcp flooding attack detection in software-defined networking using proxy-based and machine learning-based mechanisms. *Cluster Computing* **26**, 1461–1477 (2023).
32. Muthamil Sudar, K. & Deepalakshmi, P. A two level security mechanism to detect a ddos flooding attack in software-defined networks using entropy-based and c4. 5 technique. *Journal of High Speed Networks* **26**, 55–76 (2020).
33. Hossain, M. D., Inoue, H., Ochiai, H., Fall, D. & Kadobayashi, Y. Lstm-based intrusion detection system for in-vehicle can bus communications. *IEEE Access* **8**, 185489–185502 (2020).
34. Ma, H. *et al.* A gru-based lightweight system for can intrusion detection in real time. *Security and Communication Networks* **2022** (2022).
35. Bari, B. S., Yelamarthi, K. & Ghafoor, S. Intrusion detection in vehicle controller area network (can) bus using machine learning: A comparative performance study. *Sensors* **23**, 3610 (2023).
36. Nguyen, T. P., Nam, H. & Kim, D. Transformer-based attention network for in-vehicle intrusion detection. *IEEE Access* (2023).
37. Park, S. B., Jo, H. J. & Lee, D. H. G-idcs: Graph-based intrusion detection and classification system for can protocol. *IEEE Access* (2023).
38. Sharmin, S., Mansor, H., Abdul Kadir, A. F. & A. Aziz, N. Comparative evaluation of anomaly-based controller area network ids. In *Proceedings of the 2023 12th International Conference on Software and Computer Applications*, 218–226 (2023).
39. Tariq, S., Lee, S. & Woo, S. S. Cantransfer: Transfer learning based intrusion detection on a controller area network using convolutional lstm network. In *Proceedings of the 35th annual ACM symposium on applied computing*, 1048–1055 (2020).
40. Zhang, L. & Ma, D. A hybrid approach toward efficient and accurate intrusion detection for in-vehicle networks. *IEEE Access* **10**, 10852–10866 (2022).
41. Wei, J., Chen, Y., Lai, Y., Wang, Y. & Zhang, Z. Domain adversarial neural network-based intrusion detection system for in-vehicle network variant attacks. *IEEE Communications Letters* **26**, 2547–2551 (2022).
42. Desta, A. K., Ohira, S., Arai, I. & Fujikawa, K. Rec-cnn: In-vehicle networks intrusion detection using convolutional neural networks trained on recurrence plots. *Vehicular Communications* **35**, 100470 (2022).
43. Umair, M. B. *et al.* A network intrusion detection system using hybrid multilayer deep learning model. *Big data* (2022).
44. Shrestha, A. & Mahmood, A. Review of deep learning algorithms and architectures. *IEEE access* **7**, 53040–53065 (2019).
45. Luo, F. et al. In-vehicle network intrusion detection systems: a systematic survey of deep learning-based approaches. *PeerJ Computer Science* **9**, e1648 (2023).
46. Salih, A. A. et al. Deep learning approaches for intrusion detection. *Asian journal of research in computer science* **9**, 50–64 (2021).
47. Lampe, B. & Meng, W. A survey of deep learning-based intrusion detection in automotive applications. *Expert Systems with Applications* 119771 (2023).
48. Guo, Q., He, Z. & Wang, Z. Monthly climate prediction using deep convolutional neural network and long short-term memory. *Scientific Reports* **14**, 17748 (2024).
49. Yamak, P. T., Yujian, L. & Gadosey, P. K. A comparison between arima, lstm, and gru for time series forecasting. In *Proceedings of the 2019 2nd International Conference on Algorithms, Computing and Artificial Intelligence*, 49–55 (2019).
50. Yang, S., Yu, X. & Zhou, Y. Lstm and gru neural network performance comparison study: Taking yelp review dataset as an example. In *2020 International workshop on electronic communication and artificial intelligence (IWECAI)*, 98–101 (IEEE, 2020).
51. Lin, C.-B., Dong, Z., Kuan, W.-K. & Huang, Y.-F. A framework for fall detection based on openpose skeleton and lstm/gru models. *Applied Sciences* **11**, 329 (2020).

52. Khosravi, M., Parsaei, H., Rezaee, K. & Helfroush, M. S. Fusing convolutional learning and attention-based bi-lstm networks for early alzheimer's diagnosis from eeg signals towards iomt. *Scientific Reports* **14**, 26002 (2024).
53. Seabe, P. L., Moutsinga, C. R. B. & Pindza, E. Forecasting cryptocurrency prices using lstm, gru, and bi-directional lstm: A deep learning approach. *Fractal and Fractional* **7**, 203 (2023).
54. Ding, D., Zhu, L., Xie, J. & Lin, J. In-vehicle network intrusion detection system based on bi-lstm. In *2022 7th International Conference on Intelligent Computing and Signal Processing (ICSP)*, 580–583 (IEEE, 2022).
55. Zhao, H., Cheng, A., Wang, Y., Wang, S. & Wang, H. Can intrusion detection system based on data augmentation and improved bi-lstm. In *2024 IEEE Asia Pacific Conference on Circuits and Systems (APCCAS)*, 198–202 (IEEE, 2024).
56. Imrana, Y., Xiang, Y., Ali, L. & Abdul-Rauf, Z. A bidirectional lstm deep learning approach for intrusion detection. *Expert Systems with Applications* **185**, 115524 (2021).
57. Jaseena, K. U. & Kovoor, B. C. Decomposition-based hybrid wind speed forecasting model using deep bidirectional lstm networks. *Energy Conversion and Management* **234**, 113944 (2021).
58. Hossain, M. D., Inoue, H., Ochiai, H., Fall, D. & Kadobayashi, Y. An effective in-vehicle can bus intrusion detection system using cnn deep learning approach. In *GLOBECOM 2020-2020 IEEE global communications conference*, 1–6 (IEEE, 2020).
59. Ahmed, I., Jeon, G. & Ahmad, A. Deep learning-based intrusion detection system for internet of vehicles. *IEEE Consumer Electronics Magazine* **12**, 117–123 (2021).
60. Lin, H.-C., Wang, P., Chao, K.-M., Lin, W.-H. & Chen, J.-H. Using deep learning networks to identify cyber attacks on intrusion detection for in-vehicle networks. *Electronics* **11**, 2180 (2022).
61. GL. Everything you need to know about vgg16. [Accessed 16-May-2023].
62. GFG. Vgg-16. [Accessed 16-May-2023].
63. Simonyan, K. & Zisserman, A. Very deep convolutional networks for large-scale image recognition. *arXiv preprint* arXiv:1409.1556 (2014).
64. Thakur, N., Bhattacharjee, E., Jain, R., Acharya, B. & Hu, Y.-C. Deep learning-based parking occupancy detection framework using resnet and vgg-16. *Multimedia Tools and Applications* 1–24 (2023).
65. Musleh, D., Alotaibi, M., Alhaidari, F., Rahman, A. & Mohammad, R. M. Intrusion detection system using feature extraction with machine learning algorithms in iot. *Journal of Sensor and Actuator Networks* **12**, 29 (2023).
66. Anjum, A., Agbaje, P., Hounsinou, S. & Olufowobi, H. In-vehicle network anomaly detection using extreme gradient boosting machine. In *2022 11th Mediterranean Conference on Embedded Computing (MECO)*, 1–6 (IEEE, 2022).
67. Refat, R. U. D., Elkhail, A. A., Hafeez, A. & Malik, H. Detecting can bus intrusion by applying machine learning method to graph based features. In *Intelligent Systems and Applications: Proceedings of the 2021 Intelligent Systems Conference (IntelliSys) Volume 3*, 730–748 (Springer, 2022).
68. Moulahi, T., Zidi, S., Alabdulatif, A. & Atiquzzaman, M. Comparative performance evaluation of intrusion detection based on machine learning in in-vehicle controller area network bus. *IEEE Access* **9**, 99595–99605 (2021).
69. Alajlan, N. N. & Ibrahim, D. M. Tinyml: Enabling of inference deep learning models on ultra-low-power iot edge devices for ai applications. *Micromachines* **13**, 851 (2022).
70. Khandelwal, S. & Shreejith, S. A lightweight fpga-based ids-ecu architecture for automotive can. In *2022 international conference on field-programmable technology (ICFPT)*, 1–9 (IEEE, 2022).
71. Rangsikunpum, A., Amiri, S. & Ost, L. An fpga-based intrusion detection system using binarised neural network for can bus systems. In *2024 IEEE International Conference on Industrial Technology (ICIT)*, 1–6 (IEEE, 2024).

Author contributions

R.R. and A.P. conducted the experiments, R.R. and J.G. drafted the manuscript, R.R., J.G., and P.S. analyzed the results and did the manuscript proofreading. All authors reviewed the manuscript.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to P.S.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Terms and Conditions

Springer Nature journal content, brought to you courtesy of Springer Nature Customer Service Center GmbH (“Springer Nature”).
Springer Nature supports a reasonable amount of sharing of research papers by authors, subscribers and authorised users (“Users”), for small-scale personal, non-commercial use provided that all copyright, trade and service marks and other proprietary notices are maintained. By accessing, sharing, receiving or otherwise using the Springer Nature journal content you agree to these terms of use (“Terms”). For these purposes, Springer Nature considers academic use (by researchers and students) to be non-commercial.
These Terms are supplementary and will apply in addition to any applicable website terms and conditions, a relevant site licence or a personal subscription. These Terms will prevail over any conflict or ambiguity with regards to the relevant terms, a site licence or a personal subscription (to the extent of the conflict or ambiguity only). For Creative Commons-licensed articles, the terms of the Creative Commons license used will apply.
We collect and use personal data to provide access to the Springer Nature journal content. We may also use these personal data internally within ResearchGate and Springer Nature and as agreed share it, in an anonymised way, for purposes of tracking, analysis and reporting. We will not otherwise disclose your personal data outside the ResearchGate or the Springer Nature group of companies unless we have your permission as detailed in the Privacy Policy.
While Users may use the Springer Nature journal content for small scale, personal non-commercial use, it is important to note that Users may not:

1. use such content for the purpose of providing other users with access on a regular or large scale basis or as a means to circumvent access control;
2. use such content where to do so would be considered a criminal or statutory offence in any jurisdiction, or gives rise to civil liability, or is otherwise unlawful;
3. falsely or misleadingly imply or suggest endorsement, approval , sponsorship, or association unless explicitly agreed to by Springer Nature in writing;
4. use bots or other automated methods to access the content or redirect messages
5. override any security feature or exclusionary protocol; or
6. share the content in order to create substitute for Springer Nature products or services or a systematic database of Springer Nature journal content.

In line with the restriction against commercial use, Springer Nature does not permit the creation of a product or service that creates revenue, royalties, rent or income from our content or its inclusion as part of a paid for service or for other commercial gain. Springer Nature journal content cannot be used for inter-library loans and librarians may not upload Springer Nature journal content on a large scale into their, or any other, institutional repository.
These terms of use are reviewed regularly and may be amended at any time. Springer Nature is not obligated to publish any information or content on this website and may remove it or features or functionality at our sole discretion, at any time with or without notice. Springer Nature may revoke this licence to you at any time and remove access to any copies of the Springer Nature journal content which have been saved.
To the fullest extent permitted by law, Springer Nature makes no warranties, representations or guarantees to Users, either express or implied with respect to the Springer nature journal content and all parties disclaim and waive any implied warranties or warranties imposed by law, including merchantability or fitness for any particular purpose.
Please note that these rights do not automatically extend to content, data or other material published by Springer Nature that may be licensed from third parties.
If you would like to use or distribute our Springer Nature journal content to a wider audience or on a regular basis or in any other manner not expressly permitted by these Terms, please contact Springer Nature at

onlineservice@springernature.com